

CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI

THIẾT KẾ TRIỂN KHAI

TS. Đỗ Như Tài
Đại Học Sài Gòn
dntai@sgu.edu.vn

Microservices Deployments with Containers and Orchestrators

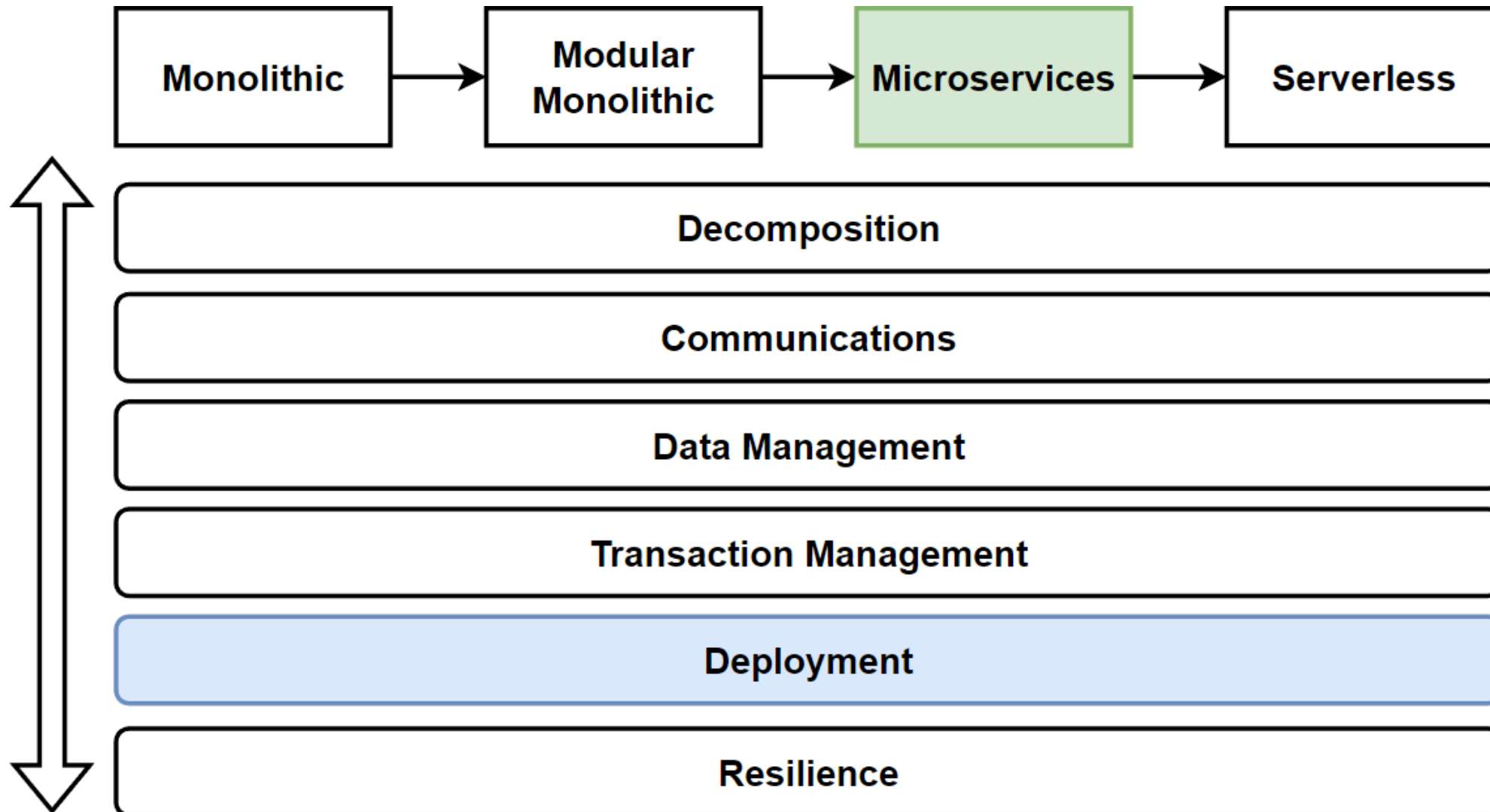
Deployment strategies; blue-green, rolling and canary deployment.

Kubernetes Patterns; Sidecar Patterns, Service Mesh Pattern

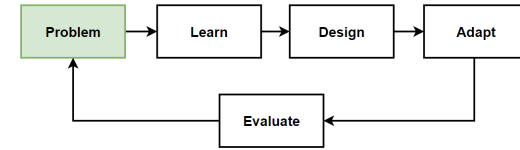
DevOps and CI/CD Pipelines

Infrastructure as code (IaC)

Architecture Design – Vertical Considerations



Problem: Deploy Microservices at Anytime with Zero-downtime and flexible scale



Problems

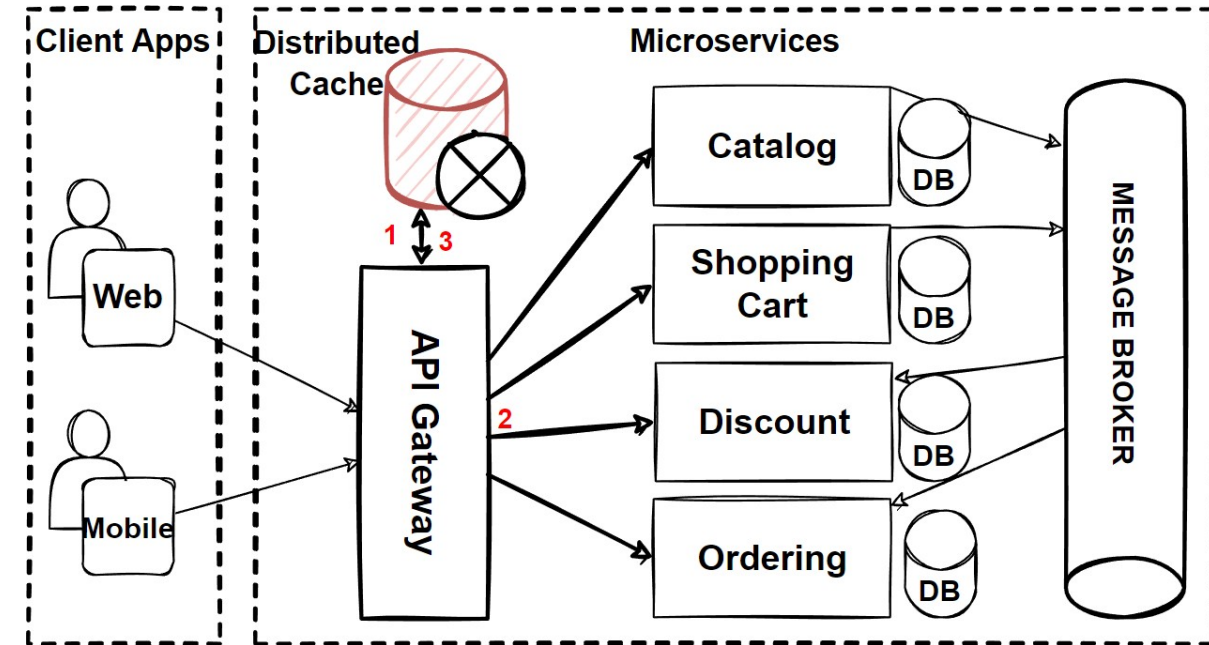
- Business teams want to add new features immediately
- Innovate and experiment with new features
- Deploy features immediately, not waiting for deployment dates.
- Flexible scale for market peak times

Considerations

- Ensure continuity of service and minimize disruption
- Allow for continuous delivery
- Support high-traffic environments

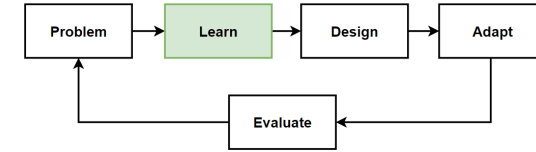
Solutions

- Containers and Orchestrators
- Deployment strategies; blue-green deployment, rolling deployment, and canary deployment.
- Kubernetes Patterns; Sidecar Patterns, Service Mesh Pattern
- DevOps and CI/CD Pipelines and Infrastructure as code (IaC)



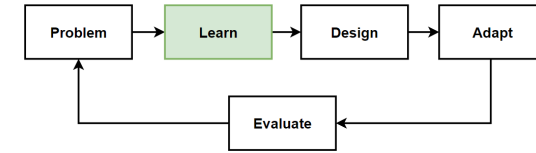
What are Containers ?

- Monolithic applications are deployed as a **single unit** and **deploy whole application** in one time.
- This caused **temporarily un-available** time of application, **needs to rollback the whole deployment** process. **Can't split modules** and scale independently.
- It solved in **microservices** architecture with **Containers** and **Orchestrators**.
- **Containers** are **package** and **distribute software applications**, makes them easy to **deploy** and **run** on **different environments**.
- Allow developers to **package an application** and its **dependencies** into a **single, self-contained** unit container images, **easily shipped** and **run** on any computer that has a container runtime.
- **Containers** are often used in the **deployment of microservices**, **independent components** can be developed, tested, and deployed separately.
- **Each microservice** is typically a **self-contained unit**, communicates with other microservices through well-defined interfaces.



What are Containers ? - 2

- **Containers** provide to **decouple applications** with their own **os**, **dependencies** and **libraries**, perfect match to microservices deployments.
- Microservices can **deployed separately** in a **container**. Each microservice can **deploy independently** with containers.
- Since **deployed microservices separate container**, we can **scale** as per their **volume of traffics**.
- **Changes** can be **applied independently** while other container stay not changed.
- **New features** can be **applied** and **rollback** very easy with **container deployments**.
- **Docker** is defacto standard for **containerization** of **microservices**.



Advantages of Containers

- **Isolation**

Each microservice runs in its own container, provides isolation from the other microservices and the host operating system.

- **Scalability**

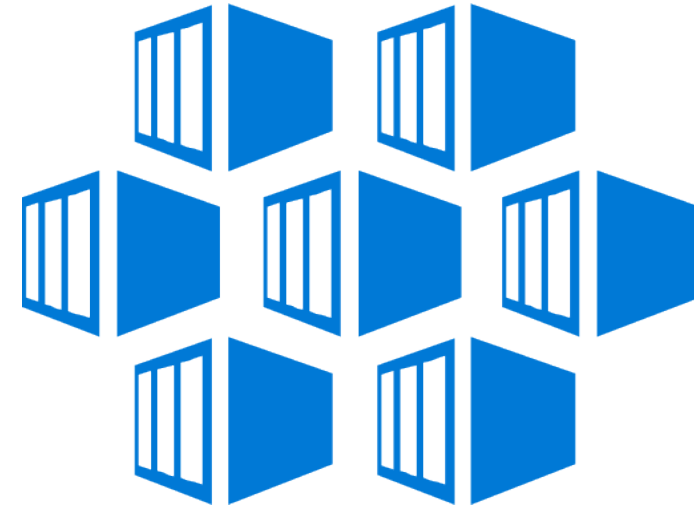
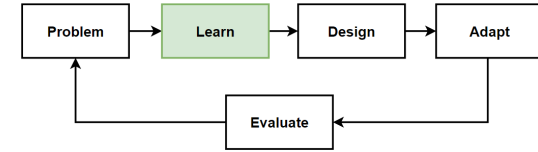
Containers make it easy to scale microservices horizontally by simply running more instances of a containerized microservice.

- **Portability**

Containers allow microservices to be easily deployed and run on any computer or cloud platform that supports container runtime.

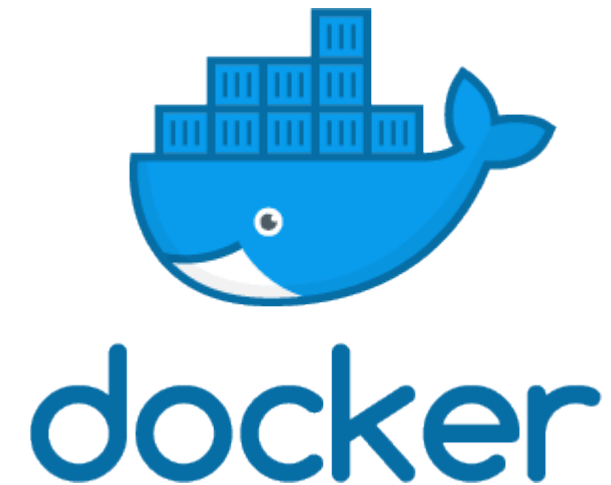
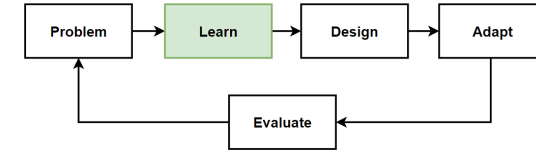
- **Resiliency**

Microservices are intended to be independently deployable and scalable. Using containers in the deployment of microservices that can be quickly started and stopped, increase the resiliency of the application.



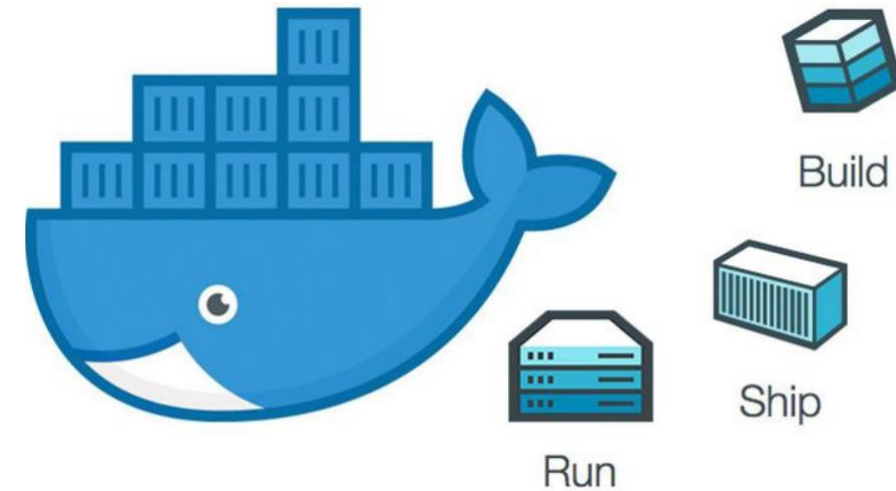
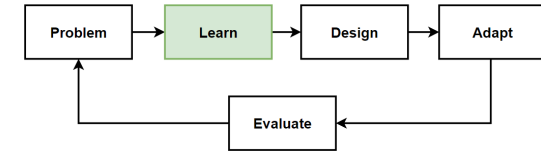
What is Docker ?

- **Docker** is an **open platform** for **developing**, **shipping**, and **running** applications.
- **Separate** your **applications** from your **infrastructure** so you can deliver software quickly.
- Advantages of Docker's methodologies for **shipping**, **testing**, and **deploying** code **quickly**.
- **Significantly reduce** the delay between **writing code** and **running** it in **production**.
- **Automating the deployment** of applications as **portable**, **self-sufficient containers** that can run on the **cloud** or **on-premises**.
- **Docker containers** can run anywhere, in your local computer to the cloud.
- **Docker image** containers can **run natively** on **Linux** and **Windows**.
- **Docker** is defacto standard for **containerization** of **microservices**.



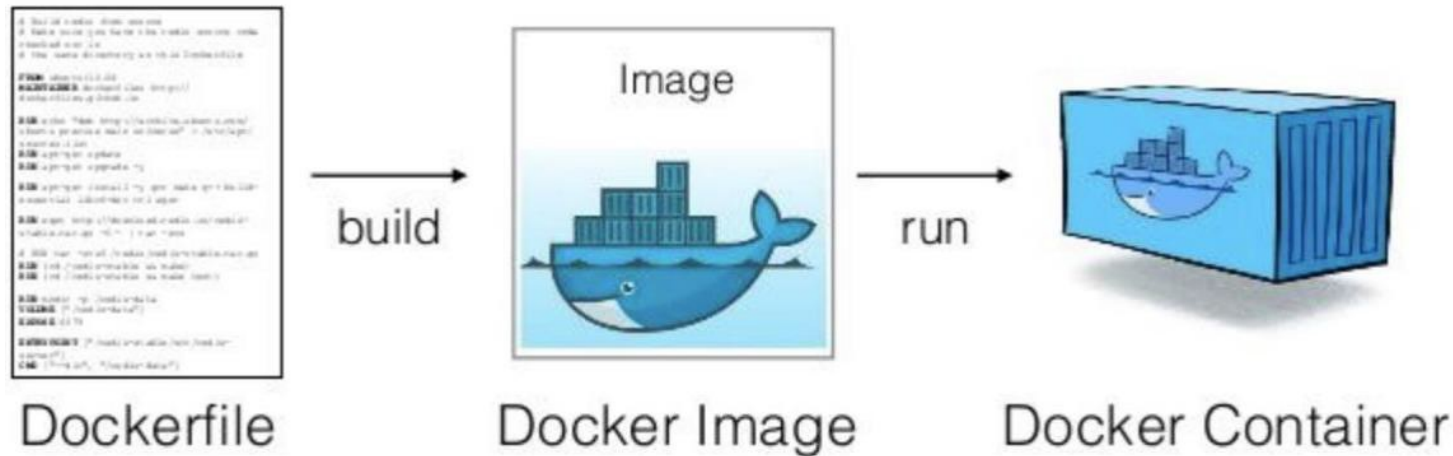
Docker Containers, Images, and Registries

- Developer **develops** and **packages** application **with** its **dependencies** into a **container image**, that is a **static representation** of the application **with** its **configuration** and **dependencies**.
- To run the application, the application's image is **instantiated to create** a **container**, which will be running on the Docker host.
- Store **images in a registry**, which is a **library of images** and is needed when deploying to production orchestrators.
- **Docker images** are **stores** a public **registry** like **Docker Hub**, **Azure Container Registry**.
- Developer **creates container** in local and **push the images** the **Docker Registry**.
- Developer **download existing image** from **registry** and create container from image in local environment.

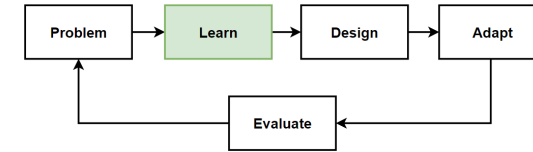


Application Containerization with Docker

1. **Write Dockerfile** for our application.
2. **Build application** with this docker file and **creates the docker images**.
3. **Run this images** on any machine and creates **running docker container** from docker image.

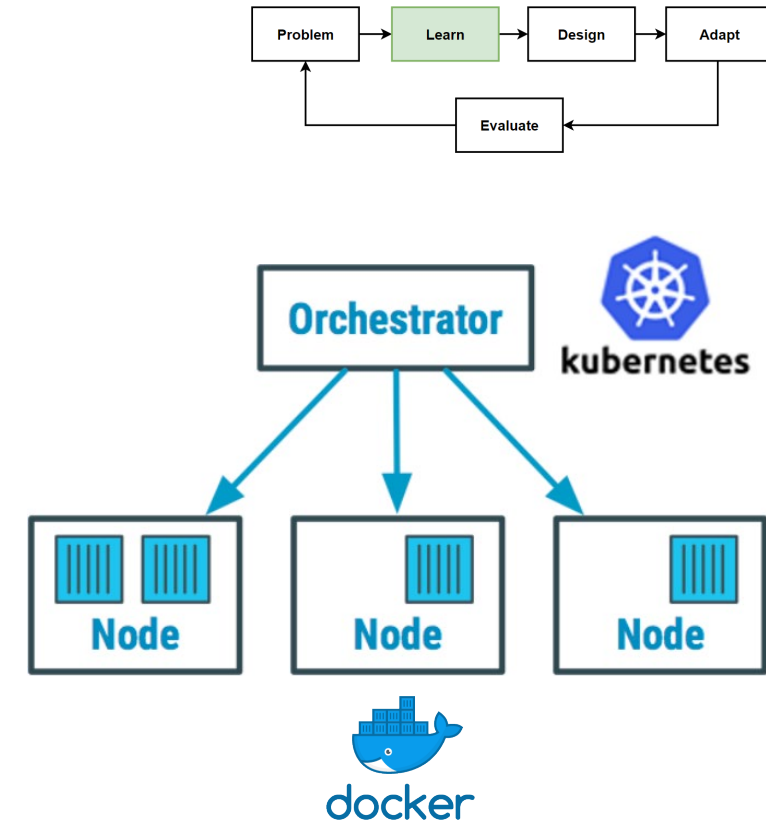


- **Orchestrating** whole **microservices** application with **Docker** and **Kubernetes**.



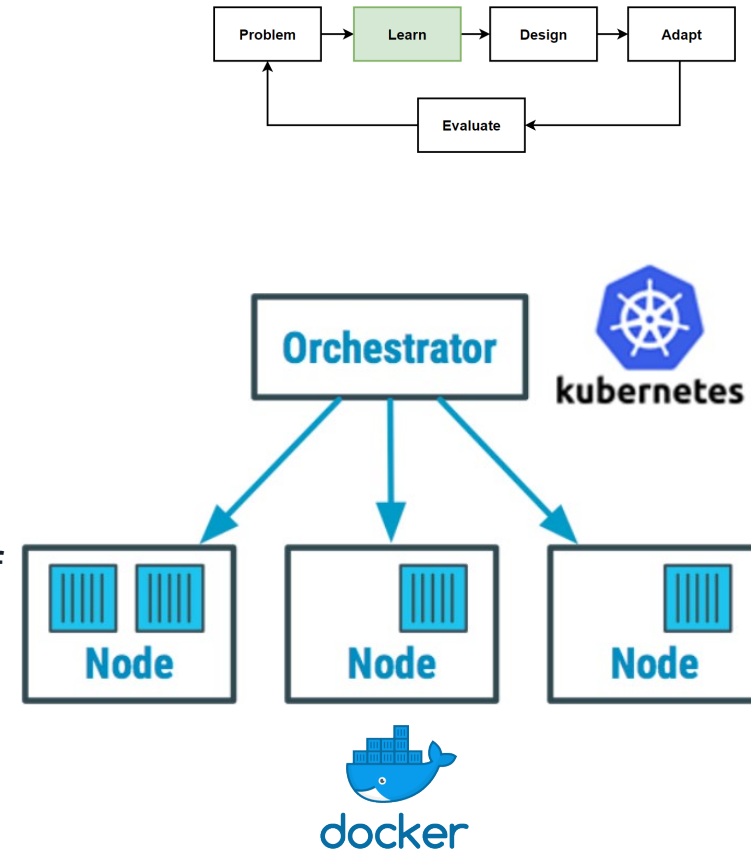
Why need Orchestrator for Containers ?

- Why we **need to orchestrate** our **containers** ?
- Think to developed and containerize our microservices and ready for shipping and deployment. Why we **need an orchestrator** for our **containers** ?
- **Deployment requirements of microservices**, think that have **hundreds of microservices**, we should ask;
 - How container instances be provisioned into cluster of multiple machines?
 - After deployment, how will containers discover and communicate with each other?
 - How can containers scale in or out on-demand and peak traffic ?
 - How do you monitor the health of each container?
 - How do you protect containers against hardware and software failures?
 - How do upgrade containers for a live application with zero downtime?
- All these questions handled by **Container Orchestrators** that automate all these concerns.
- These task **can not be managed by manually** administrated for thousands of independently deployed containers.
- **Containerized services require automated management.**

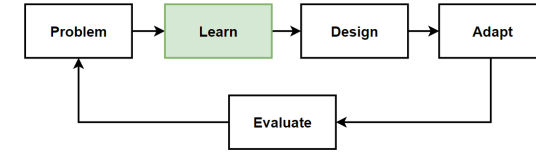


What is Container Orchestrator ?

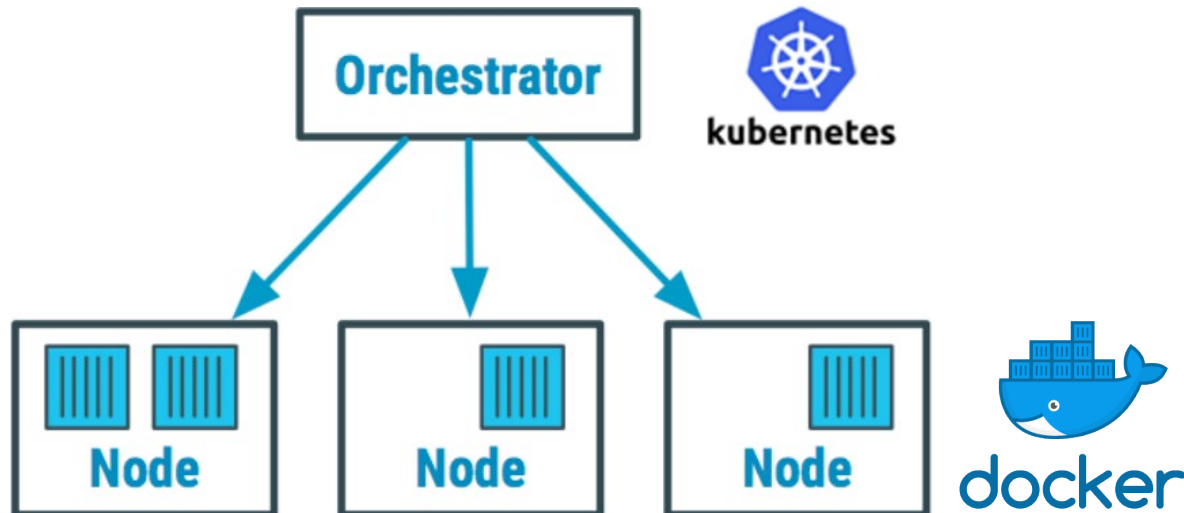
- **Container orchestrators** are **manage** and **automate** the **deployment**, **scaling**, and management of containerized applications.
- Enable to **run** and **manage microservices-based applications** in an efficient and scalable way by **abstracting the underlying infrastructure** and handling tasks such as **resource allocation**, **load balancing**, and **monitoring**.
- Microservices architectures, **containers need to orchestrate** to manage lots of container in your application cluster.
- **Orchestrators automates** the **deployment**, **scaling**, and **operational concerns** of containerized workloads across clusters.
- Container orchestrator **automates the deployment** and **management** of these microservices, making it **easier to scale** and **update application** as needed.
- There are several popular **container orchestrators** available, including **Kubernetes**, **Docker Swarm**, and **Apache Mesos**.



Application Containerization and Orchestration



1. **Package** each of **microservices** into a **container image** and **push** it to a **container registry**.
2. **Then** use the **container orchestrator** to **deploy** the **microservices** to a cluster of nodes.
3. **Orchestrator** will handle tasks such as **scheduling containers** onto nodes, **monitoring** the health of the containers, and providing **self-healing** capabilities if a container fails.



Benefits of Using Container Orchestration

- **Automation**

orchestrator can automate the deployment and management of your microservices.

- **Scalability**

orchestrator can automatically scale your microservices up or down as needed to meet changing demand.

- **High availability**

orchestrator can provide self-healing capabilities to ensure that your microservices remain available even if individual containers fail.

- **Improved resource utilization**

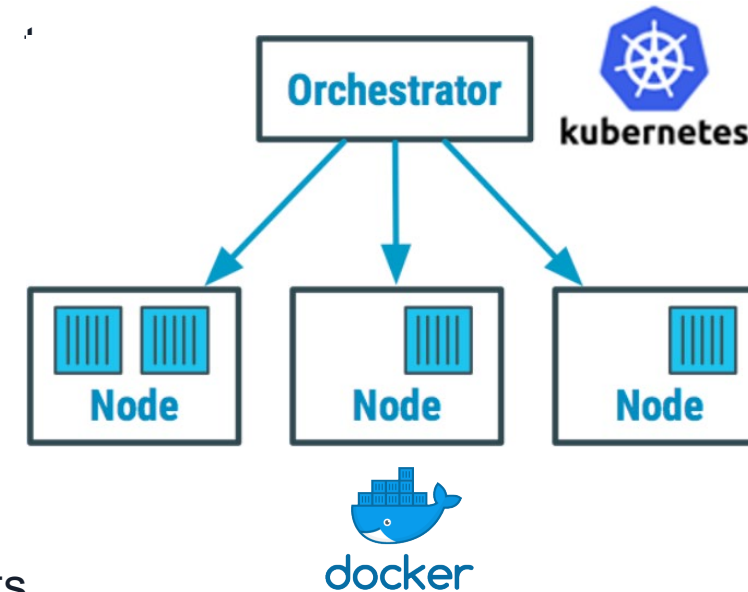
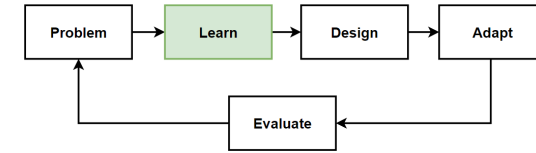
Orchestrator can optimize the use of resources in your cluster.

- **Portability**

Containerized applications can be easily moved between different environments and platforms.

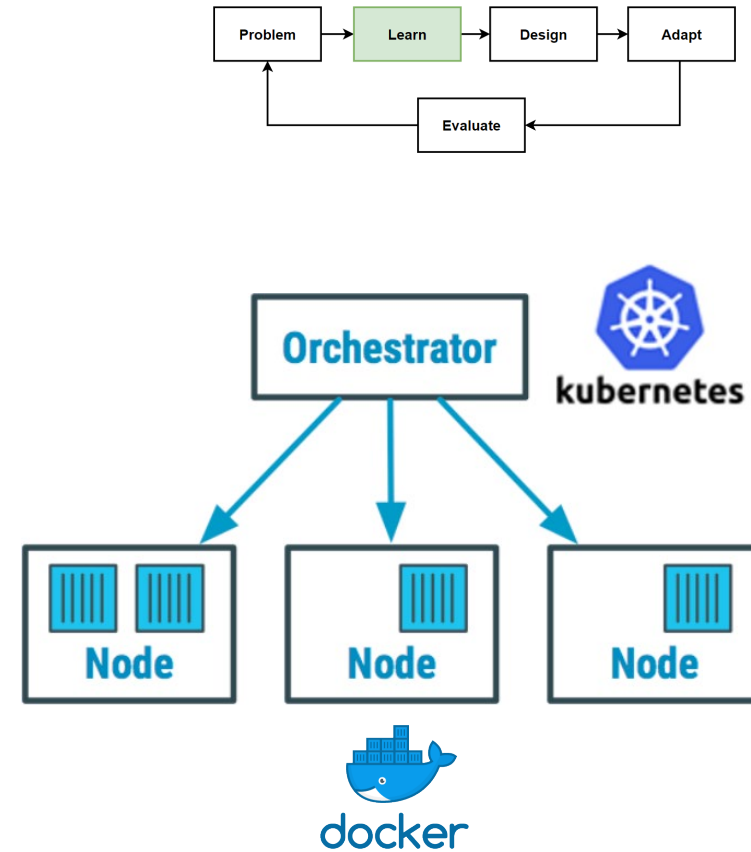
- **Security**

orchestrators provide security features such as role-based access control and network segmentation.



What is Kubernetes ?

- **Kubernetes** (also known as **k8s** or "**kube**") is an open source **container orchestration platform**.
- **Automates** many of the manual processes involved in **deploying**, **managing**, and **scaling containerized** applications.
- Developed by **Google** and is now maintained by the **Cloud Native Computing Foundation (CNCF)**.
- **Kubernetes** is a **portable, extensible, open-source platform** for managing containerized workloads and services, both **declarative configuration** and **automation**.
- It has a large, **rapidly growing ecosystem**. Kubernetes services, support, and tools are widely available.
- **Create cluster groups** of hosts running Linux containers, Kubernetes helps you easily and efficiently manage those clusters.
- Kubernetes is designed to provide a **platform-agnostic** way to **manage containerized applications at scale**.



Benefits of Kubernetes

- **Self-healing**

Kubernetes can automatically restart containers that fail, ensuring that your microservices remain available even if individual containers go down.

- **Automatic scaling**

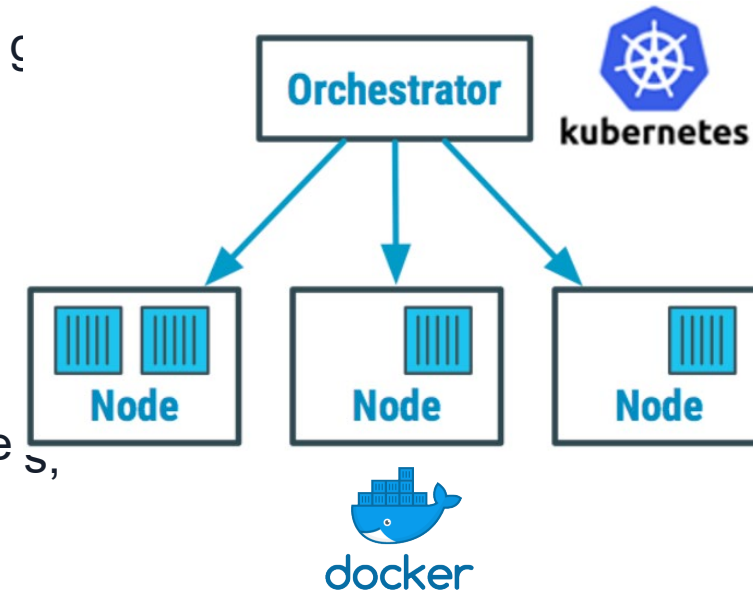
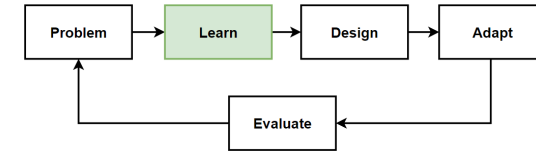
Automatically scale your microservices up or down as needed to meet changing demand, making it easier to handle sudden spikes in traffic.

- **Load balancing**

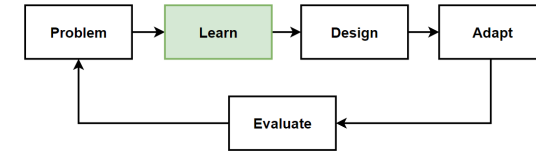
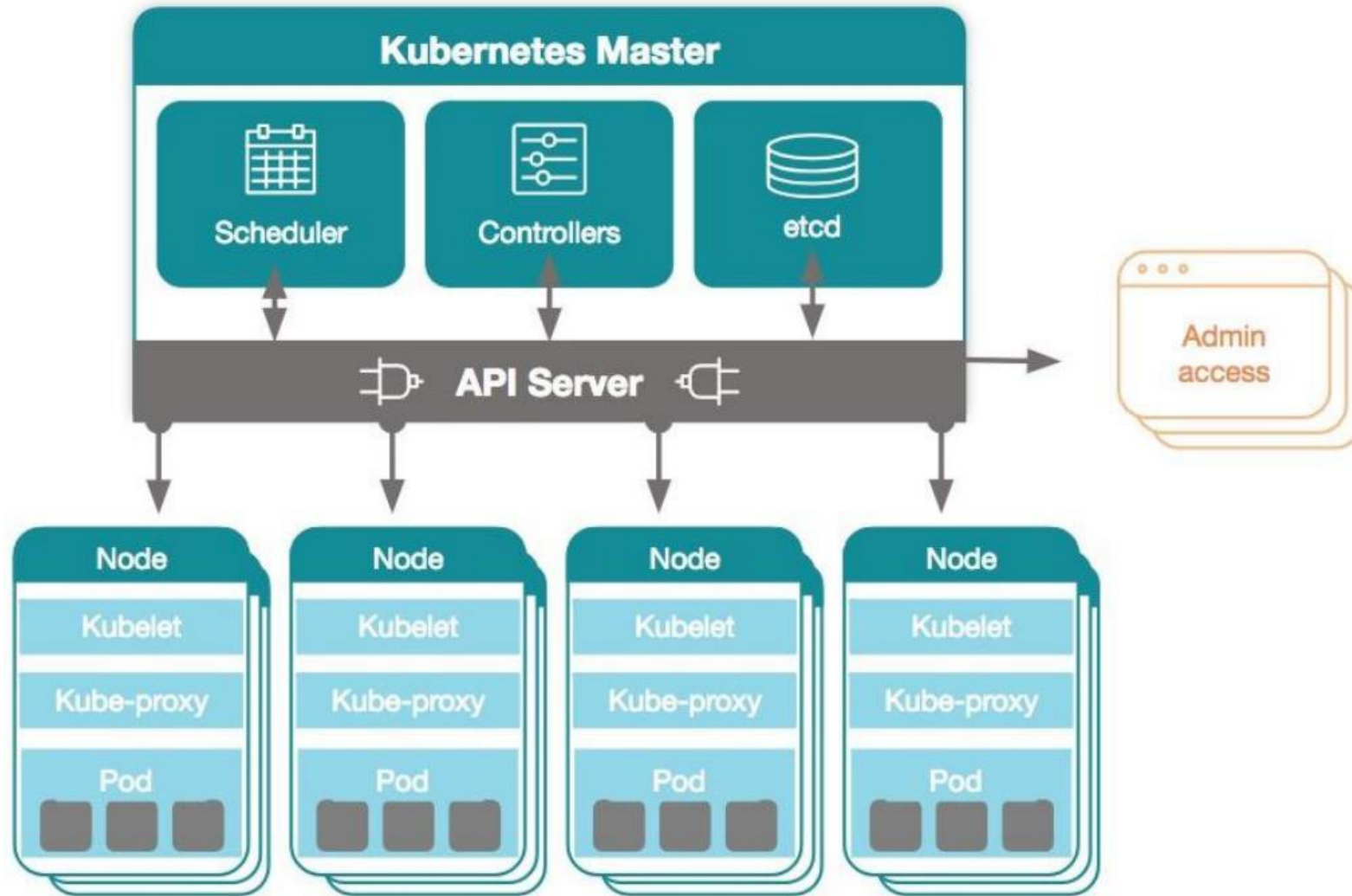
Automatically distribute incoming traffic across multiple instances of a microservice.

- **Declarative configuration**

Allows to specify the desired state of your microservices using configuration files, easy to automate the deployment and management of your applications.



Kubernetes Architecture



Kubernetes Components

- **Pods**

Pods are the smallest deployable units of computing, that you can create and manage in Kubernetes. Pods stores and manage our docker containers.

- **ReplicaSet**

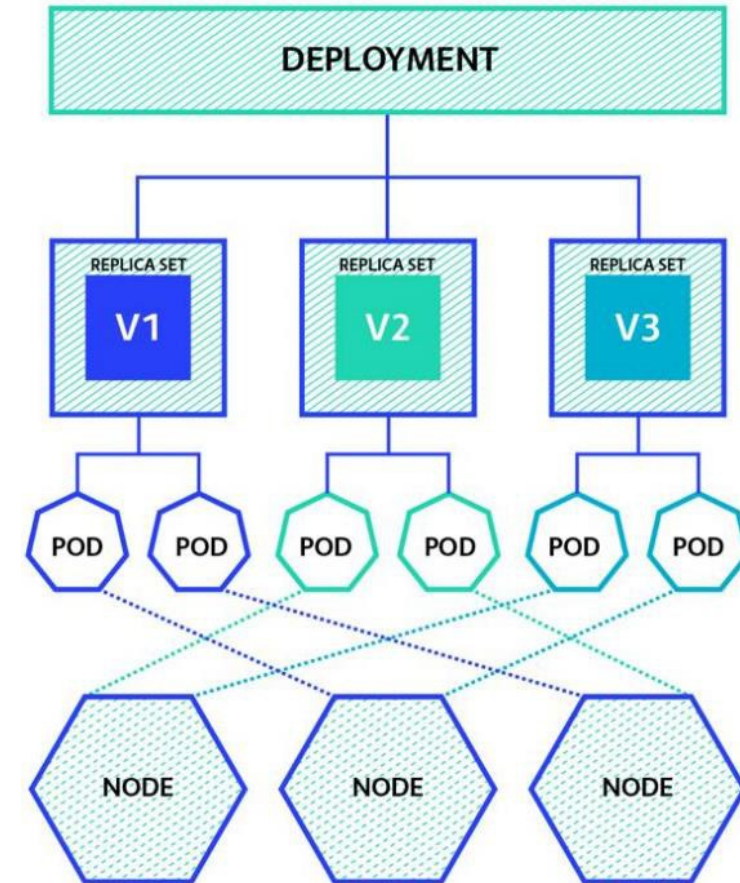
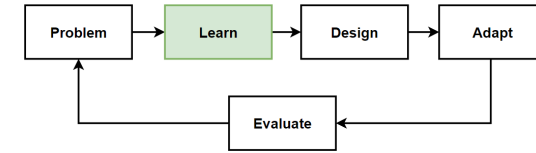
Maintain a stable set of replica Pods running at any given time. Used to guarantee the availability of a specified number of identical Pods.

- **Deployments**

Provides declarative updates for Pods and ReplicaSets. Describe a desired state in a Deployment, and the Deployment Controller changes the actual state to the desired state at a controlled rate.

- **Deployments** are an **abstraction of ReplicaSets**, and **ReplicaSets** are an **abstraction of Pods**.

- **Pods should not created directly**, if needed, Deployment objects should be created.



Kubernetes Components - 2

- **Service**

Expose an application running on a set of Pods as a network service. Kubernetes gives Pods their own IP addresses and a single DNS name for a set of Pods, and can load-balance across them.

- **ConfigMaps**

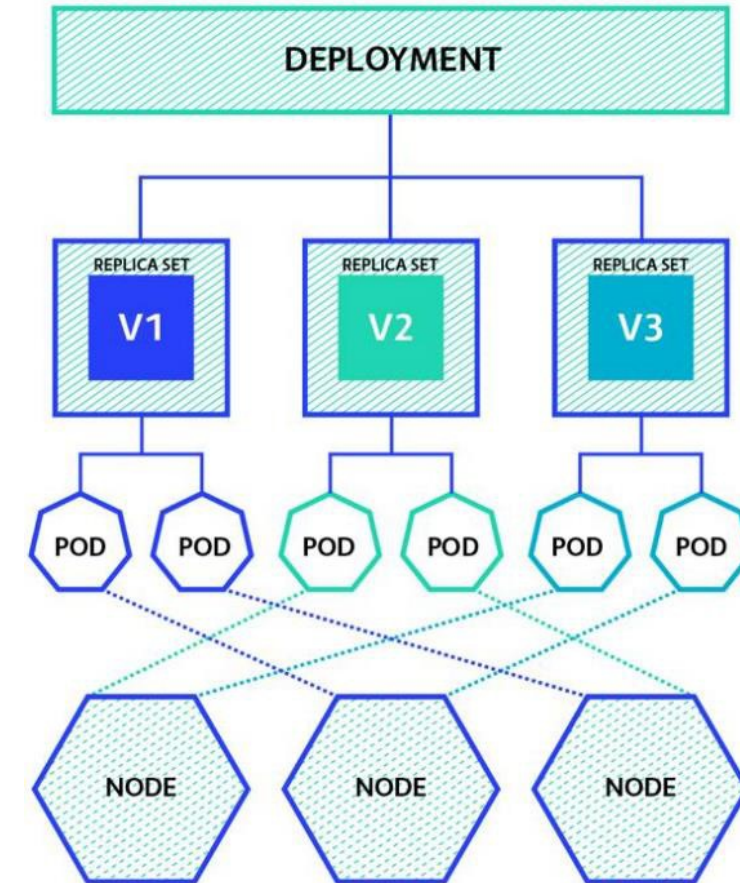
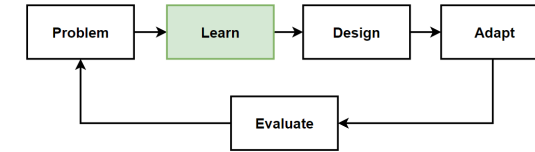
API object used to store non-confidential data in key-value pairs. Pods can consume ConfigMaps as environment variables, command-line arguments, or as configuration files in a volume.

- **Secrets**

Store and manage sensitive information, such as passwords, OAuth tokens, and ssh keys.

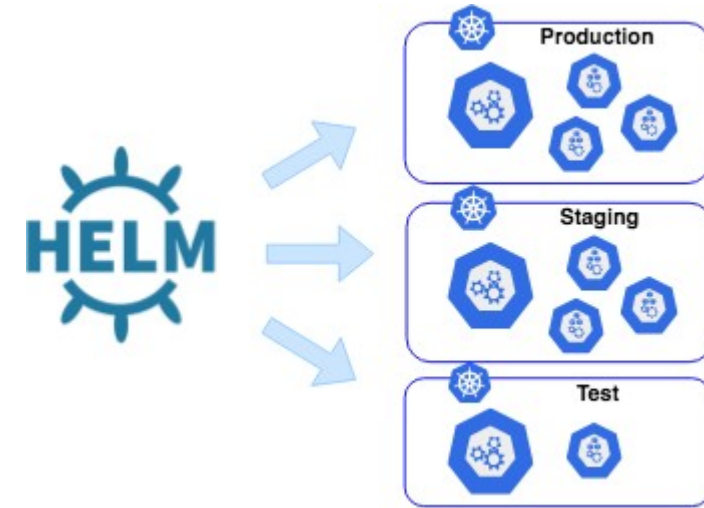
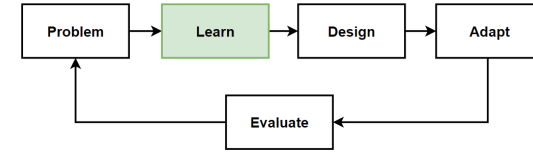
- **Volumes**

Persistent storage location that can be mounted into a pod. Volumes can be used to store data that needs to persist across container restarts or be shared between multiple containers in a pod.



Helm Charts with Kubernetes for Microservices Deployments

- **Helm** is a **package manager** for **Kubernetes** that makes it easier to manage and deploy applications in a Kubernetes cluster.
- **Helm uses** a **packaging format** called **charts**, which are **collections of files** that describe the various components of an application (such as pods, services, and deployments)
- It allows you to **automate the deployment** of **complex applications** in a **Kubernetes cluster**.
- **Instead of manually creating** and **managing** each component of your application separately, can use a **single Helm chart** to define all of the components and their relationships.
- This makes it **much easier to manage** and **update** your **application** over time.



Benefits of Helm Charts with Kubernetes for Microservices Deployments

- **Reusability**

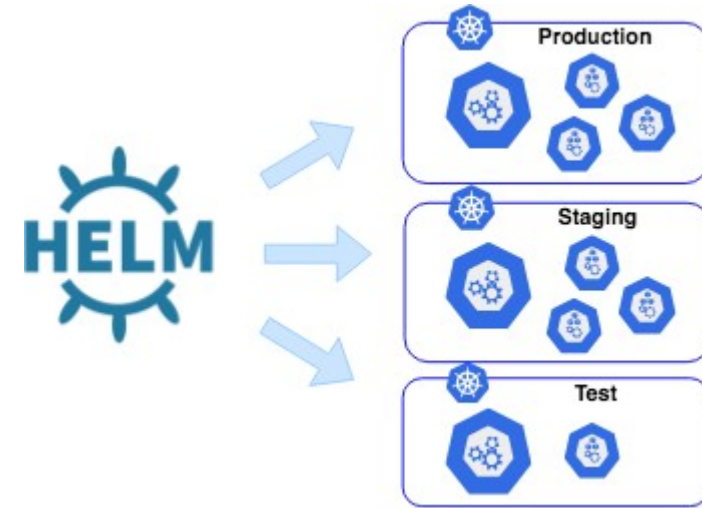
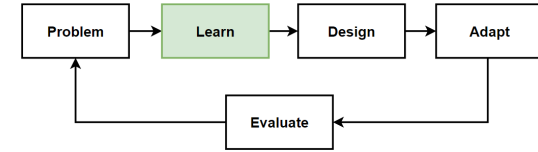
Helm charts can be shared and reused across multiple clusters, making it easy to deploy the same application in different environments.

- **Version control**

Allowing to track changes to your application over time and roll back to previous versions if needed.

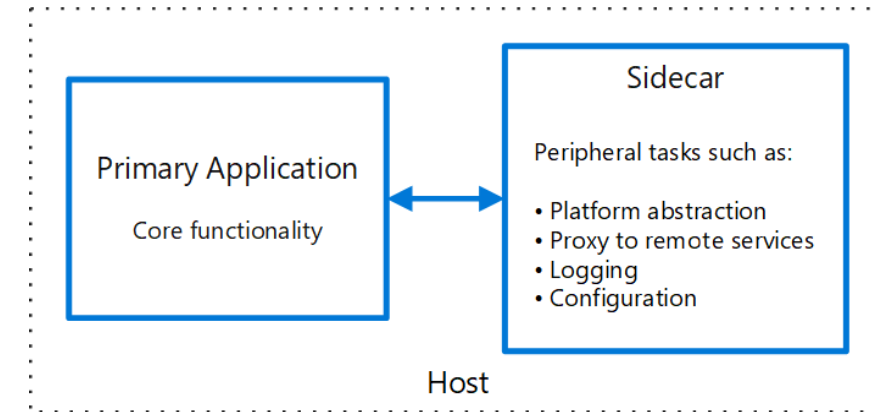
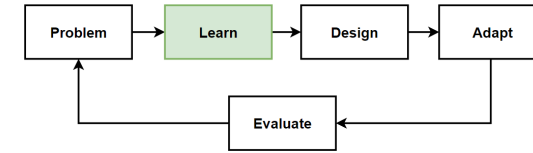
- **Customization**

Allow you to override the default settings for a chart and tailor it to your specific needs.



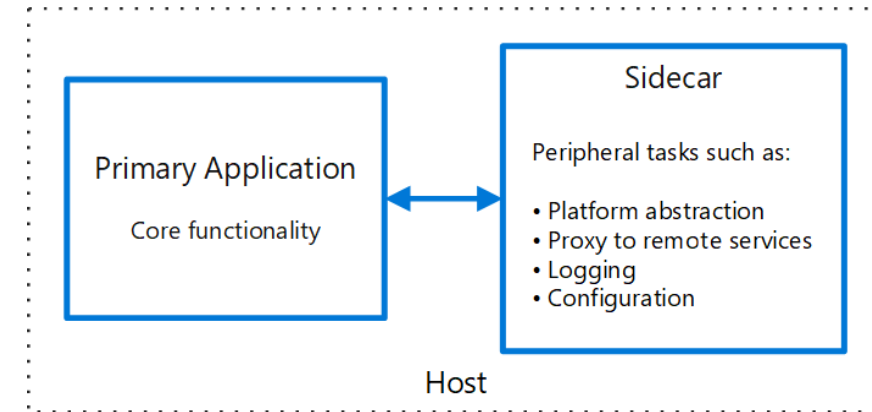
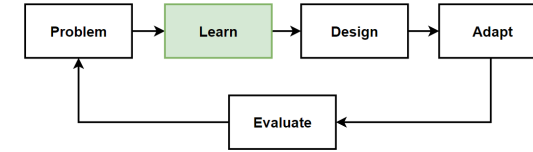
Sidecar Pattern

- **Sidecar pattern** is a design pattern in which a **helper container** is run alongside the **main container in a pod**.
- **The Sidecar container performs cross-cutting tasks** that are related to the main container.
- **Common use case** for the **sidecar pattern** is to add **logging** or **monitoring** functionality to a container.
- **A sidecar container** that runs a **logging agent** that sends application logs to a **central logging server**, and also **provide to monitoring** the **health** of the **main container**.
- **These cross-cutting tasks** can be implemented as **separate services** with **sidecar pattern** such as **monitoring**, **logging**, **configuration**, and **networking** services.
- The sidecar container and the main container **share the same network namespace**, that allows the sidecar container to **access** and **manipulate** the **data produced** by the **main container** as needed.



Benefits of Sidecar Pattern

- Allows to **add functionality** to a **container** without modifying the container itself.
- Useful if you want to **add logging** or **monitoring functionality** to an existing container image that you do not control.
- Allows you to **decouple the main container** from the **sidecar functionality**, easier to update or replace the sidecar without affecting the main container.
- Allows to run **multiple containers in a pod** that share the same network namespace, making it **easier to communicate** between them.
- Allows to **add common functionality** to multiple microservices **without having to modify the microservices** themselves.



Drawbacks of Sidecar Pattern

- **Increased complexity**

Adding an additional layer of complexity to your deployment, more difficult to understand and troubleshoot issues that arise.

- **Increased resource usage**

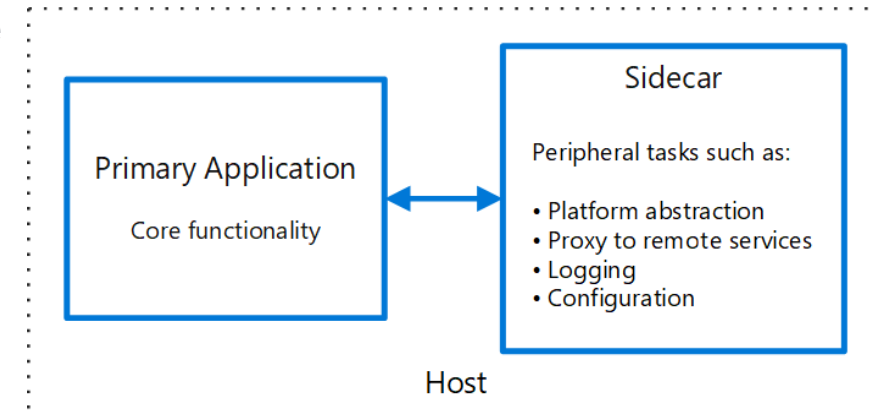
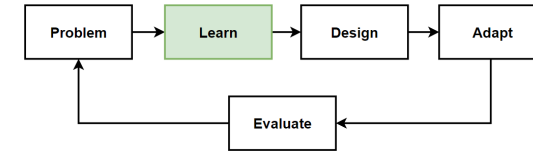
Running an additional container in a pod will increase the resource usage of the pod.

- **Decreased performance**

Pod can potentially decrease the performance of the pod, as the sidecar container will be competing for resources with the main container.

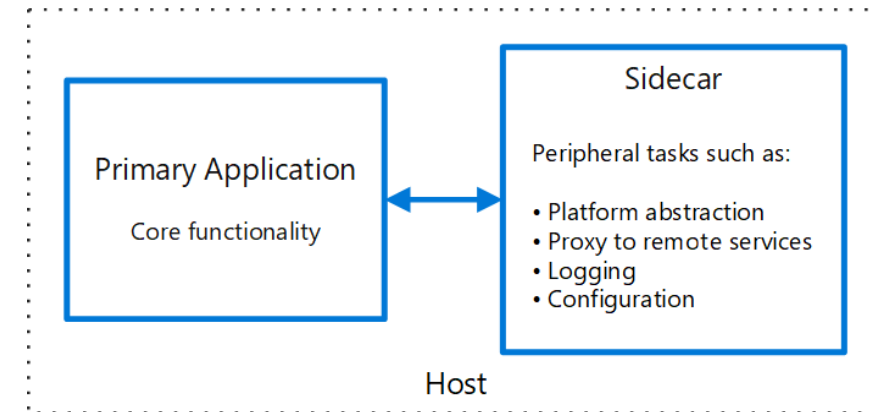
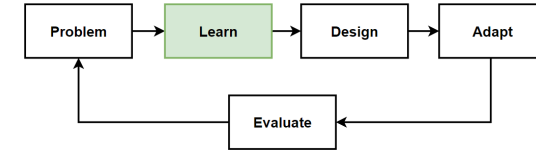
- **Limited flexibility**

Can be inflexible in some cases, as it requires that the main container and the sidecar container run in the same pod.



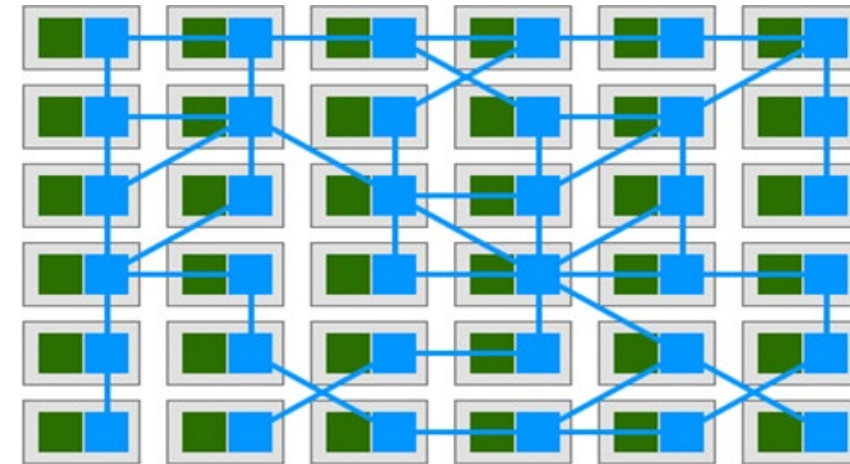
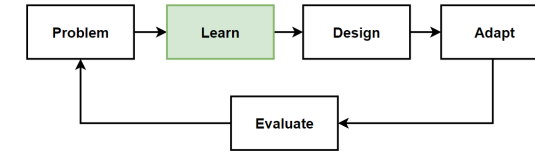
When to use Sidecar Pattern

- When you want to **add functionality** to an **existing container** image
- When you want to **decouple the main container** from the additional functionality
- When you want to **run multiple containers in a pod** that need to communicate with each other
- When you want to **add common functionality** to multiple microservices



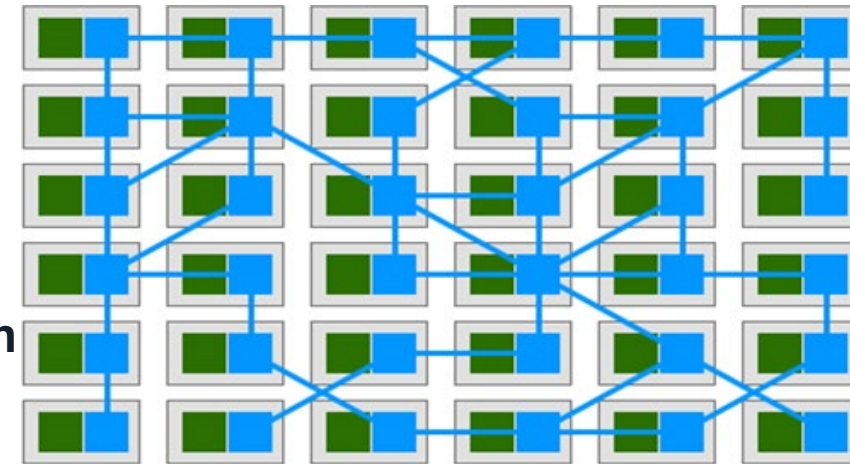
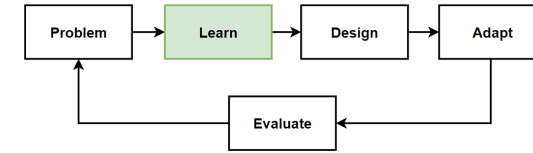
Service Mesh Pattern

- **Service mesh pattern** is **managing the communication** between **microservices** in a distributed system.
- Designed to provide a **uniform way to route traffic** between microservices, handle **load balancing**, and **monitor the healths**.
- Consists of a **set of proxy servers (sidecars)** that are deployed alongside the microservices. Proxy servers **handle the communication** between the microservices.
- **Responsible** for tasks such as **routing requests**, **load balancing**, and **monitoring the health** of the system.
- **Abstract away the complexities** of managing communication between microservices.
- **Instead of** having to **manage** these details at the **application level**, use the **service mesh** to handle them **automatically**.
- **Popular Service mesh** implementations, including **Istio** and **Linkerd**, set up and manage a service mesh in a **Kubernetes cluster**.



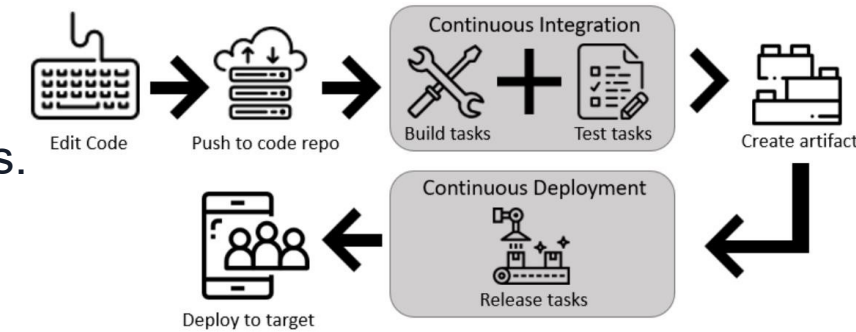
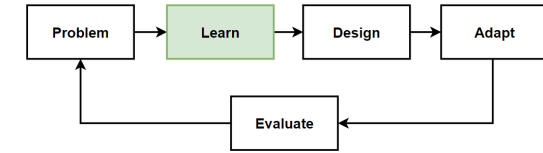
When to use Service Mesh Pattern

- When you want to **abstract away the complexities of managing communication** between microservices.
- When you want to **centralize the management** of communication between microservices.
- When you want to **add features** such as **load balancing**, **traffic management**, and **monitoring** to your microservices.
- **Using a service mesh**, can add these features to your microservices **without having to modify the microservices** themselves.
- **Service mesh pattern** is a useful tool for **managing the communication** between microservices in a distributed system.
- **Build and maintain complex microservices-based** systems by abstracting away the complexities of managing communication between the microservices.

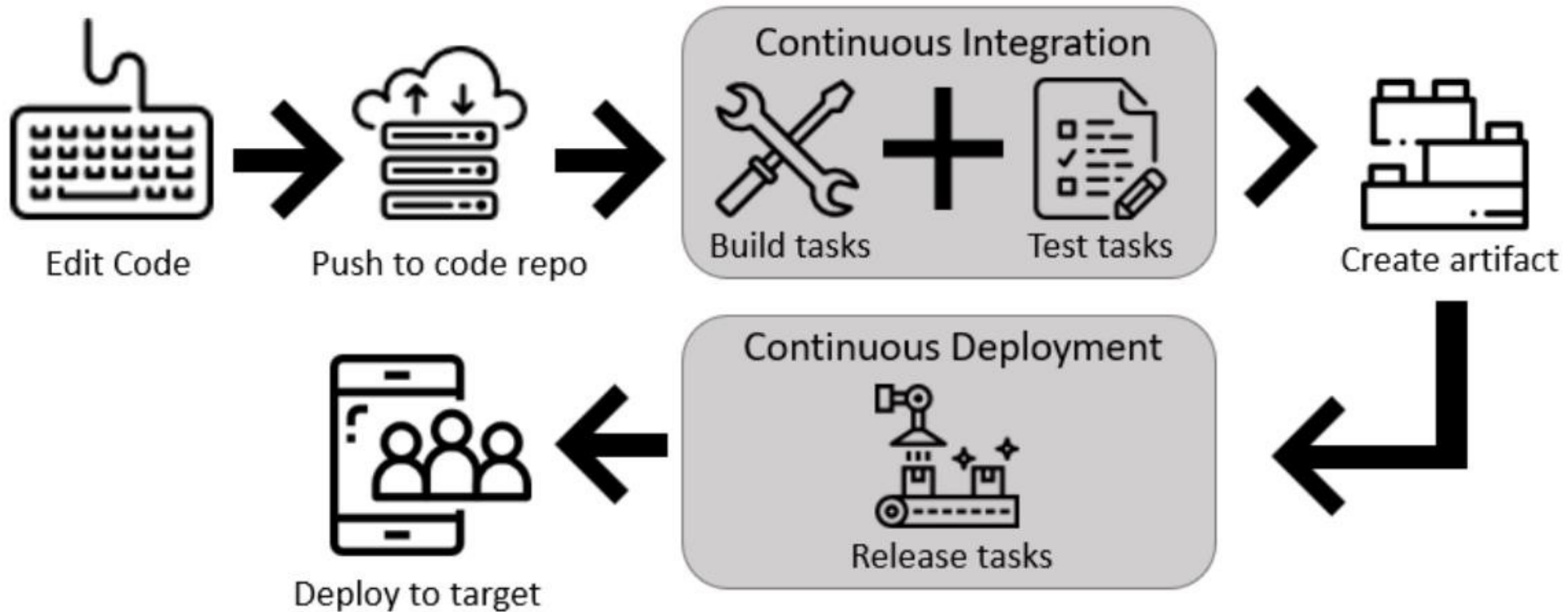
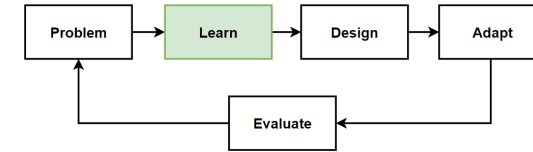


Devops and CI/CD Pipelines

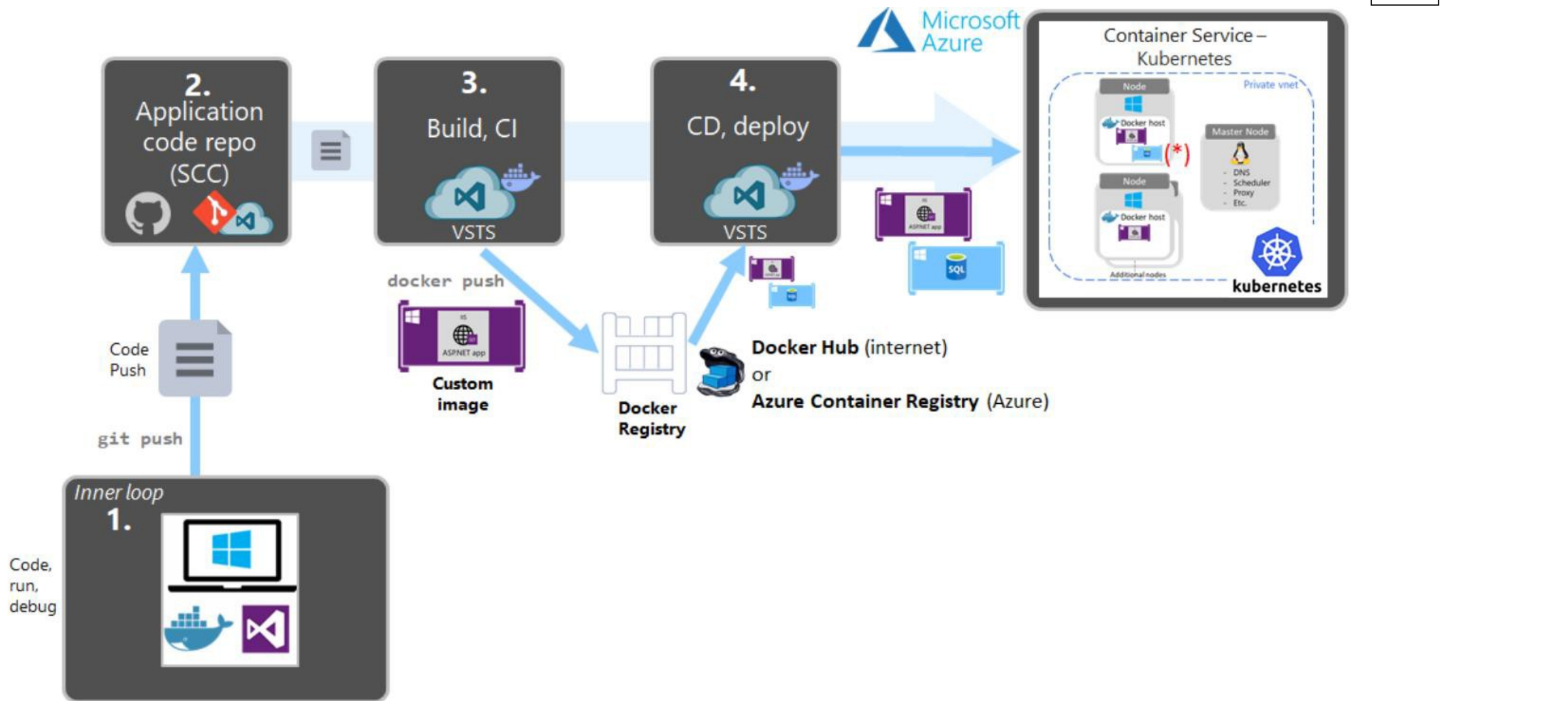
- **DevOps** is to **improve** the **efficiency** and **speed** of software **development** and **deployment** by **automating** and streamlining the processes involved.
- **Continuous Integration / Continuous Delivery (CI/CD) pipeline.**
- **CI/CD pipeline** is a **series of automated processes** that are used to **build**, **test**, and **deploy** software.
- **For microservices** deployments, a **CI/CD pipeline** can be used to **automate the process** of **building**, **testing**, and **deploying** microservices.
- **CI/CD pipeline** to **automatically build** and **test new versions** of a microservice, and then **deploy** the **new version** to a staging or production environment.
- **CI/CD pipeline** is to **enable** teams to **quickly** and **reliably deliver** changes to microservices **with zero-downtime**.
- **Deploy updates** to **individual microservices** as needed rather than having to deploy updates to the entire system at once.
- **Automate** and **streamline** the process of **building**, **testing**, and **deploying** microservices.



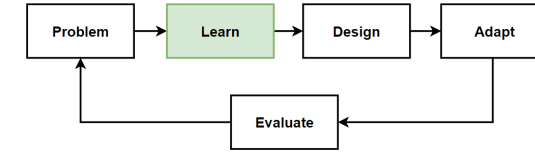
CI/CD Pipeline Steps for Microservices Deployments



CI/CD Tools for Microservices Deployments



Deployment Strategies for Microservices



- **Blue-green deployment**

Deploying updates to a new set of microservices (the "green" deployment), while the old version of the microservices (the "blue" deployment) remains running.

- **Rolling deployment**

Deploying updates to a subset of the microservices at a time, and then rolling the updates out to the rest of the microservices over time.

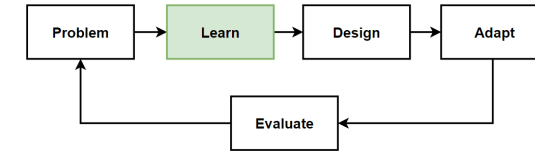
- **Canary deployment**

Deploying updates to a small subset of the microservices, and then gradually rolling the updates out to the rest of the microservices over time.

- **A/B testing**

Deploying updates to a subset of the microservices, and then comparing the performance of the updated microservices with the performance of the unmodified microservices.

Infrastructure as Code (IaC)



- **Infrastructure as a codebase** that can be **managed** and **versioned** in the same way as **application code**.
- **IaC** is to **enable teams to manage their infrastructure** in a more **automated** and **repeatable way**.
- **IaC** can be used to **automate** the **process of deploying** and **managing** the **infrastructure** needed to run the microservices.
- I.e. define the **infrastructure** needed to run your **microservices** in a **Kubernetes cluster**, including the **pods**, **services**, and **other resources** required.
- **Terraform** is a tool that allows you to define and manage infrastructure as code.
- **Ansible** is a tool that allows you to automate the deployment and management of infrastructure.
- They can be used to **define the infrastructure needed** to run your **microservices** in a **Kubernetes cluster**.
- **IaC** is a useful tool for **automating** the process of **deploying** and **managing** the **infrastructure**.
- **Teams** can **manage** their **infrastructure** in a more **repeatable** and **automated way**, making it easier to **update** and **maintain** their **microservices** over time.

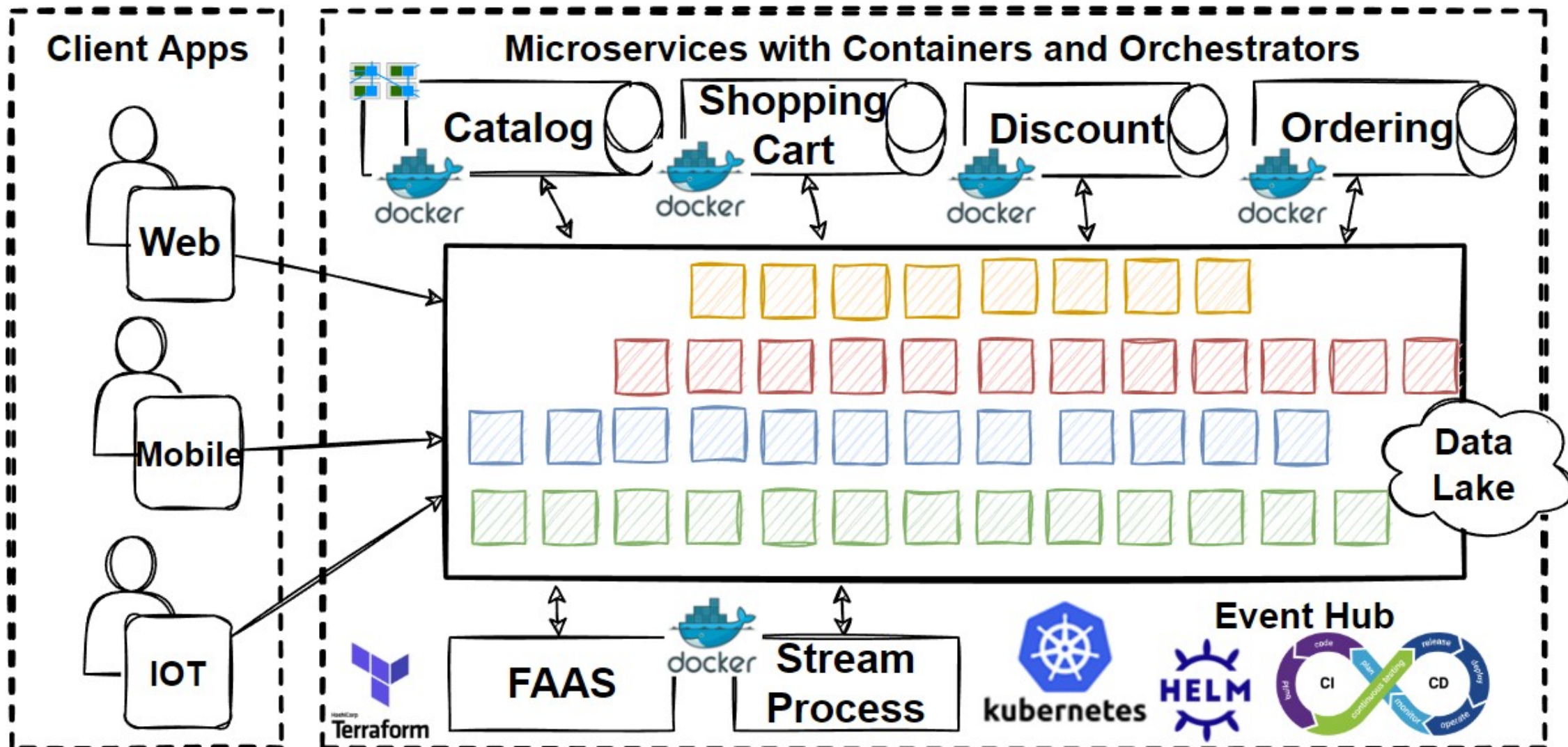
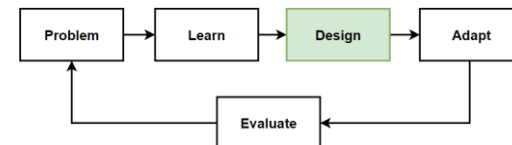
Before Design – What we have in our design toolbox ? - Old

Architectures	Patterns&Principles	Microservices Caching	Non-FR	FR
• Microservices Architecture • Event-Driven Microservices Architecture	• The Database-per-Service Pattern • Polygot Persistence • Decompose services by scalability • The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns • Microservices Data Management Patterns • Event-Driven Architecture • Microservices Distributed Caching	• Caching Strategies • Cache Invalidation • Cache Hit - Cache Miss • Cache-Aside Pattern	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

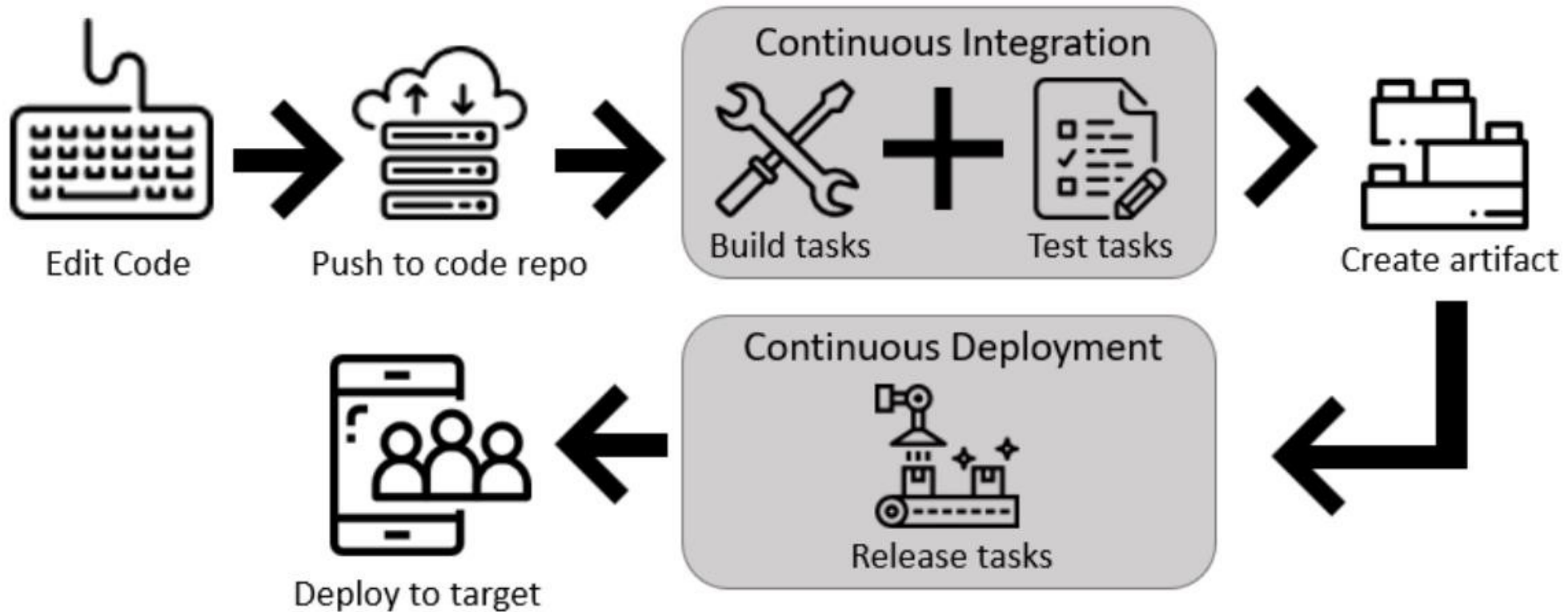
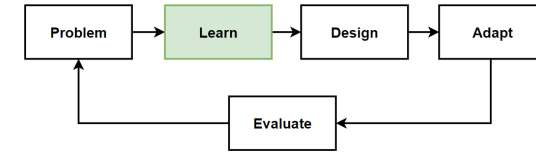
Before Design – What we have in our design toolbox ? - New

Architectures	Patterns&Principles	Microservices Deployment	Non-FR	FR
• Microservices Architecture • Event-Driven Microservices Architecture	• The Database-per-Service Pattern, Polygot Persistence, Decompose services by scalability, The Scale Cube • Microservices Decomposition Pattern • Microservices Communications Patterns • Microservices Data Management Patterns • Event-Driven Architecture • Microservices Distributed Caching • Microservices Deployments with Containers and Orchestrators	• Docker and Kubernetes Architecture, Helm Charts • Kubernetes Patterns; Sidecar Patterns, Service Mesh Pattern • DevOps and CI/CD Pipelines • Deployment Strategies; Blue-green, Rolling, Canary and A/B Deployment. • Infrastructure as code (IaC)	• High Scalability • High Availability • Millions of Concurrent User • Independent Deployable • Technology agnostic • Data isolation • Resilience and Fault isolation	• List products • Filter products as per brand and categories • Put products into the shopping cart • Apply coupon for discounts • Checkout the shopping cart and create an order • List my old orders and order items history

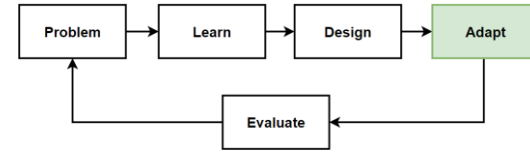
Microservices using Containers and Orchestrators



CI/CD Pipeline Steps for Microservices Deployments



Adapt: Microservices w/ Containers and Orchestrators



Frontend SPAs

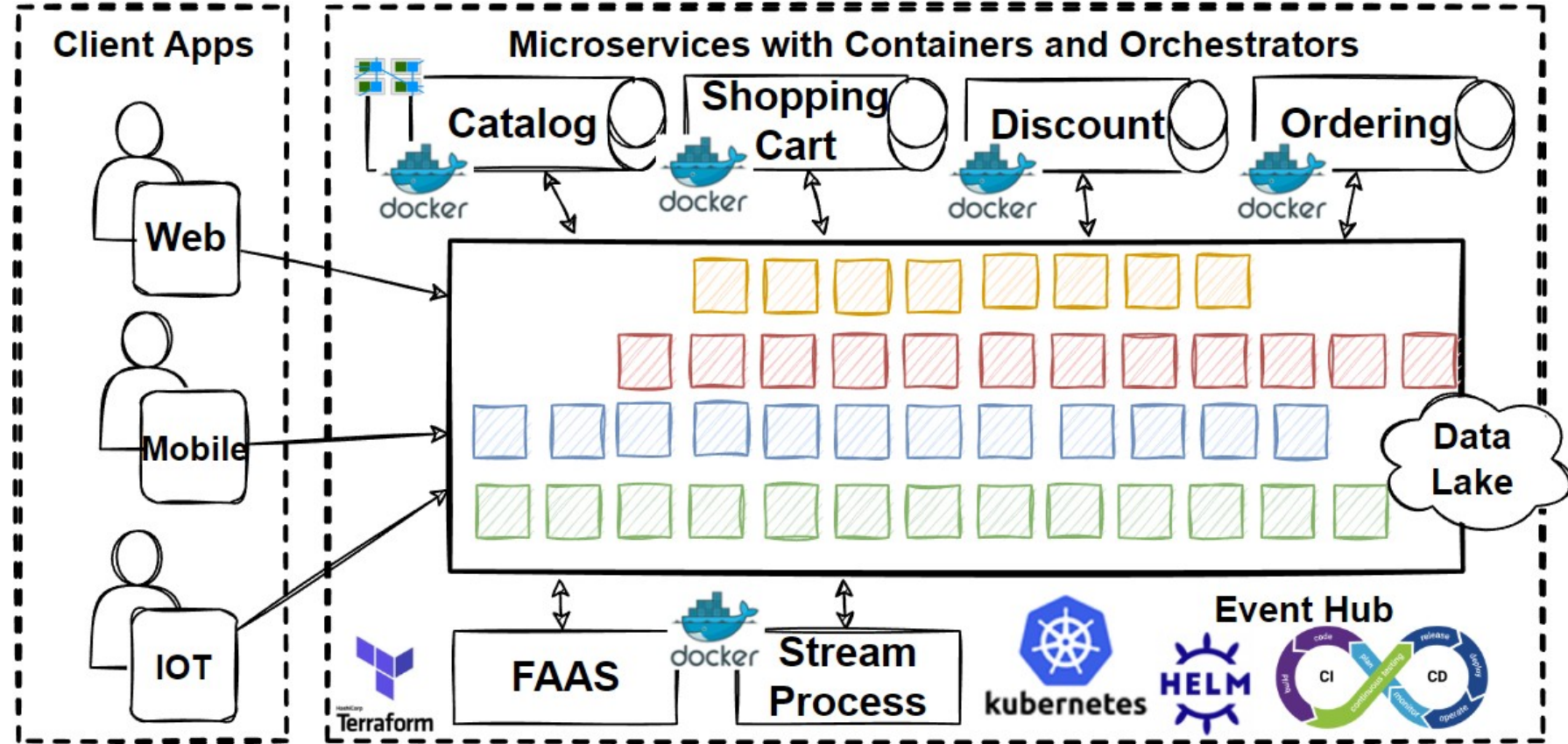
- Angular
- Vue
- React

API Gateways

- Kong Gateway
- Express Gateway
- Amazon AWS API Gateway

Backend Microservices

- Java – Spring Boot
- .Net – Asp.net
- JS – NodeJS



Container Tools

- Docker
- Containerd
- Podman

Orchestrator Tools

- Kubernetes
- Helm Charts

Cloud Orchestrator

- AKS, EKS, GKS
- AWS Elastic Beanstalk
- Azure Container Apps

Service Mesh

- Istio
- Linkerd

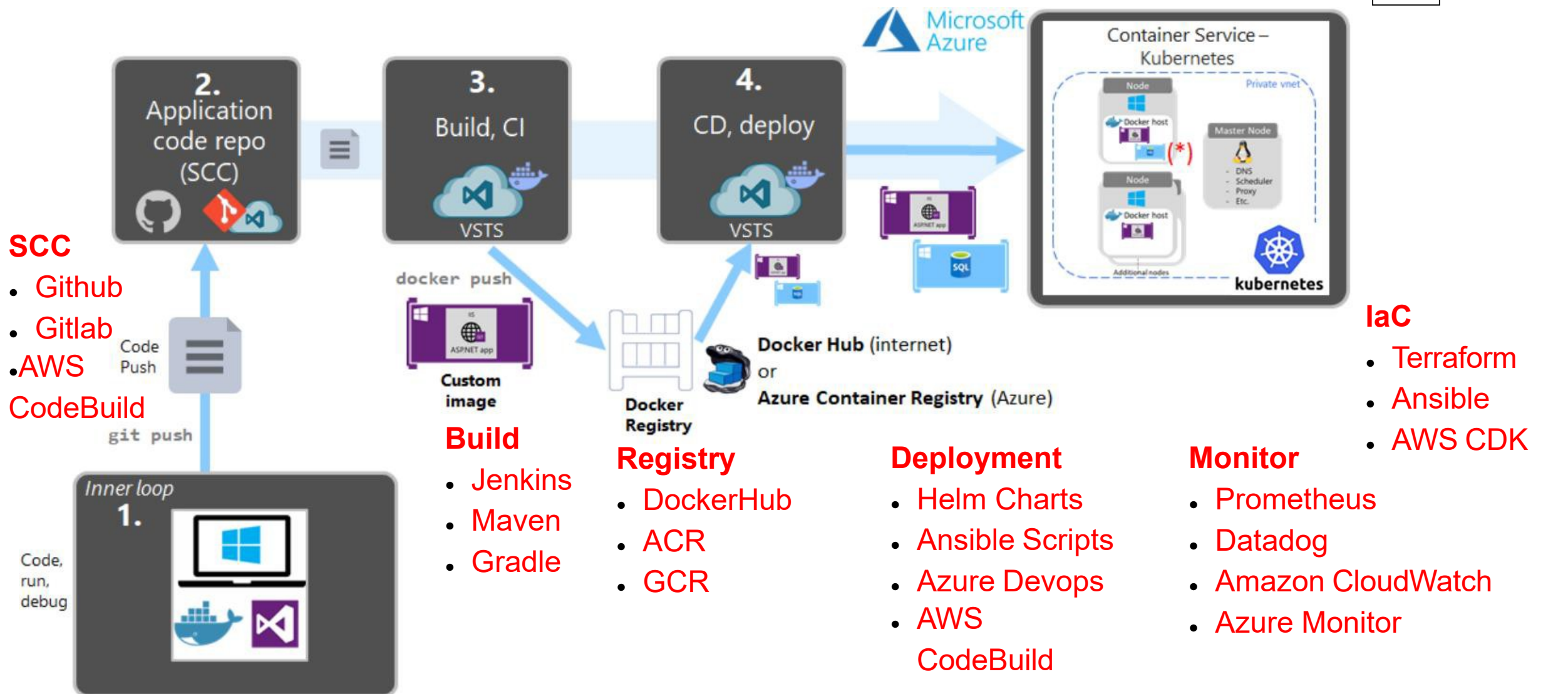
IaC

- Terraform
- Ansible

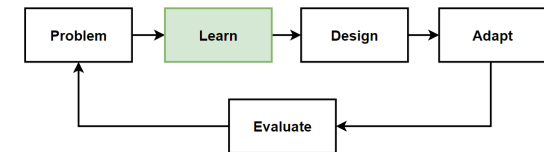
Cloud IaC

- AWS CDK

CI/CD Tools for Microservices Deployments

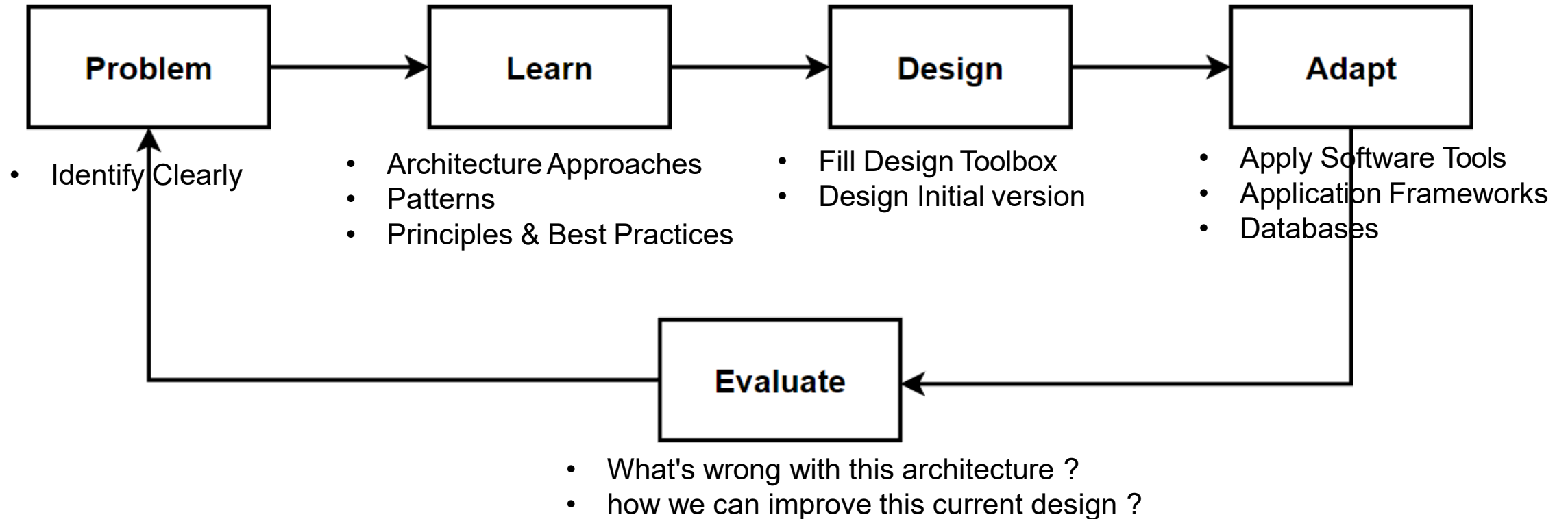


How many concurrent request can accommodate our design ?

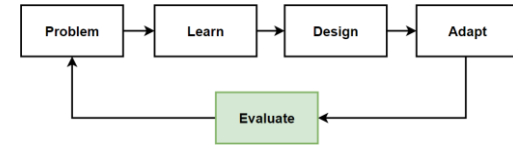


Concurrent Users	Requests/second	Latency (Expected)
2K	0.5K	
20K	12K	
100K	80K	
500K	300K	<= 2 sec

Way of Learning – The Course Flow



Evaluate: Microservices Distributed Caching

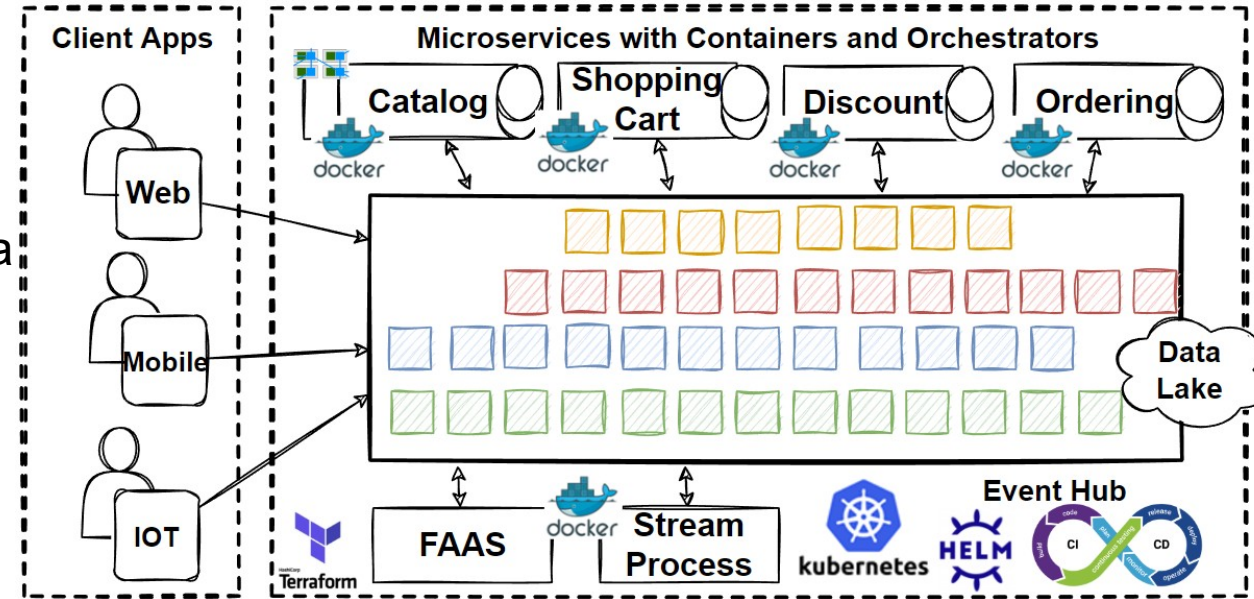


Benefits

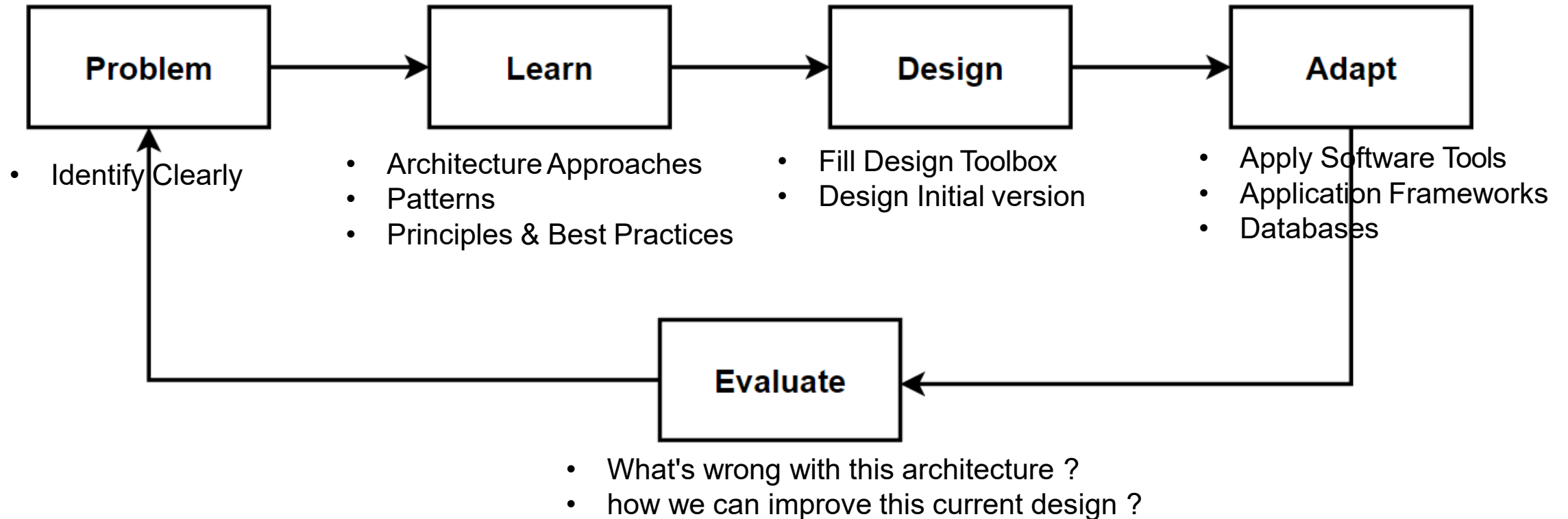
- **Portability**; Containers package applications and their dependencies into a single unit
- **Isolation**; Containers provide isolated environments for each microservice
- **Scalability**; Kubernetes can automatically scale containerized microservices up or down on demand
- **Observability**; platforms provide extensive monitoring and logging capabilities.

Drawbacks

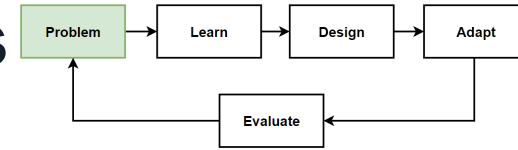
- **Complexity**; complex to set up and manage.
- **Resource overhead**; require a significant amount of resources to run CPU, memory.
- **Performance overhead**; can add some overhead to the performance
- **Dependency on external tools**; like Docker and Kubernetes



Way of Learning – The Course Flow



Problem: Fault tolerance Microservices able to remains operational for any failures or disruptions



Problems

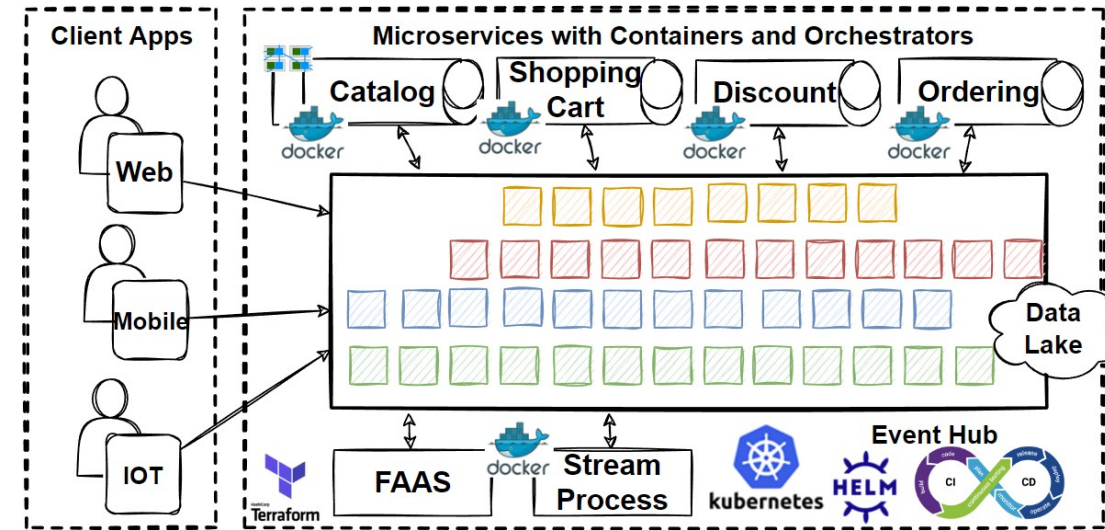
- Business teams able to deploy new features immediately to compete the market
- Due to heavy traffic, microservices got exceptions and broke some major use cases that cause lost revenue
- Create big crisis to customer loyalty

Considerations

- Deploy new features immediately without affecting the system general behaviors
- System should recover from failures and provide ability of a system to withstands.
- Architecture should be designed to be resilient

Solutions

- Microservices Resilience and Fault Tolerance
- Microservices Observability with Distributed Logging and Distributed Tracing
- Microservices Health Monitoring



**CẢM ƠN ĐÃ CHÚ Ý
LẮNG NGHE!**